



Oss Ouahdi

Senior Product Marketing Manager assignment · Prepared for RevenueCat · September 2026

RevenueCat for Product Teams

How I refreshed the /for-product page: process, AI use and assumptions

Live page <https://revenuecat-for-product-nu.vercel.app/>
Hero variant B2 <https://revenuecat-for-product-nu.vercel.app/?v=b2>
GitHub repo <https://github.com/oss1010/revenuecat-for-product>
Walkthrough video [Link to come](#)

Contents

- | | | | |
|---|--|---|---|
| 1 | Summary | 5 | Architecture and copy |
| 2 | Framing and research | 6 | Build and verification |
| 3 | Positioning, and every direction I killed | 7 | What I'd do next |
| 4 | The system: five agents and a context file | | Appendix: tools, assumptions, sources |

1 Summary

The diagnosis. The current page was written for a RevenueCat that took monetization off a PM's plate. Its promise was "You ship what matters, we handle monetization." The product has since outgrown that promise. Paywalls, Experiments with predicted 12-month LTV, web funnels and web billing now put monetization in the PM's hands. The page still describes the earlier product, and so do competitors' comparison pages: one lists four capabilities as missing from RevenueCat that RevenueCat now ships.

The positioning. Monetization is a product decision, and RevenueCat lets product managers make it like one. That principle organizes the page but never appears on it. The page is built on three jobs, drawn from the seven pain points PMs face: change it without a release, learn what users value, and fit the offer to every user, on a foundation that handles the rest. Revenue is the outcome, not the PM's job title.

The page. The hero reads "Build a subscription experience worth renewing." Eight section headlines tell the whole argument on their own. Five dynamic modules, including an experiment simulator where judging by conversion and judging by predicted 12-month LTV pick different winners.

How I worked. Claude as a strategy partner, Claude Code as the builder, five custom review agents, and a public repo where every decision is logged.

14

positioning directions
tested, 1 kept

19

hero lines considered,
1 kept

24

agent reviews run

56

claims failed by the
audit and fixed

Three kinds of callout run through this doc, because the collaboration went in every direction:

- 👤 **I challenged the AI** where I overrode a recommendation
- ⬆️ **The AI challenged me** where it pushed back and was right
- ✓ **An agent caught it** where a review agent found what neither of us had

- 1 Research**
RevenueCat's site, seven feature pages, five competitors
- 2 Positioning**
14 directions, 1 kept
- 3 System**
5 agents, 1 context file, 1 pre-commit hook
- 4 Architecture**
8 sections, 5 dynamic modules
- 5 Copy**
19 hero lines, 1 kept, 1 test variant
- 6 Build**
8 phases, static HTML, no framework
- 7 Verification**
a red team against the current page, a final 48-claim sweep, 390px first

2 Framing and research

What I read first. RevenueCat's homepage, seven feature pages, the changelog and docs for what's actually shipped and when, the pricing page, and the State of Subscription Apps 2026 report for plausibility checks. Then five competitors' pages, plus the two alternatives PMs actually weigh: building in-house and native store tools.

The competitive scan

	How it pitches product teams	Evidence tier	Signature claim	Scale
Superwall	A dedicated PM page: own the monetization roadmap end to end, without an app release or an engineering queue	Owned (dedicated PM page)	Bills only on revenue through its own paywalls	\$1.6B+ revenue, 10,000+ apps

	How it pitches product teams	Evidence tier	Signature claim	Scale
Adapty	Financial decisions on accurate revenue analytics. No PM page	Claimed (homepage section: testing for revenue, not just conversions)	12-month LTV and revenue prediction	Large, self-reported
Purchasely	The conversion problem is the whole first session, not one screen	Claimed	Onboarding journeys and personalized paywalls	Enterprise and media customers
Qonversion	A generic PM page	Owned , weakly	No-code A/B testing	\$1B+ served revenue
Apphud	Positioned as the RevenueCat alternative	Claimed	Revenue data accuracy	14,000+ apps, \$1B+ tracked, self-reported
In-house	No vendor fee, full control	n/a	"We already have engineers"	The default
Native store tools	Free and built in	n/a	Store-level experiments	Universal
RevenueCat today	"You ship what matters, we handle monetization"	n/a	n/a	146K+ apps, \$17B+

What the scan showed. The category has converged on one promise to PMs: change the paywall without engineering, test it, read the analytics. That promise is now table stakes. RevenueCat's edge is not a unique feature. It is being the category leader with the most complete platform, a 12-month forecast inside the experiment, and scale no challenger matches.

The lesson: where a claim lives matters

Where the claim appears	What it means	Does it block RevenueCat from claiming it?
Dedicated persona page or homepage headline	Owned. They own that message to that audience	Yes. Don't lead with the same frame
Homepage section or feature page	Claimed. Strong but contestable	Only if RevenueCat can't prove it more completely
Docs, changelog, help center	Parity. The capability exists, it isn't a position	No

I challenged the AI

When the competitor agent found revenue-based winners buried in two competitors' docs, the AI concluded RevenueCat should retreat to the one claim nobody else could make: "grow revenue on the system that already runs it." I rejected it. A capability in someone's documentation is parity, not positioning, and a category leader shouldn't position like a challenger. We wrote the owned, claimed and parity rule into the agent so it wouldn't make the same mistake twice.

3 Positioning, and every direction I killed

Positioning took five rounds. Each round killed something for a specific reason, and the reasons are more useful than the directions.

Round	Directions tested	What killed them	Who killed them
1	Monetization as a product surface; "speed is table stakes, knowing what worked isn't"; "your app store isn't your only store"	The AI recommended a hybrid: one as the spine, one as proof, one as a chapter. Three decisions wearing one coat	I did
2	"You own the number, now own the controls"; "change the paywall today, know what it's worth in 12 months"; "conversion is the wrong number"	The first was on Superwall's PM page almost word for word. The second merged Superwall's speed claim with Adapty's forecast claim	The competitor scan
3	"Decide on money inside the experiment"; "grow revenue on the system that already runs it"	Revenue winners inside tests turned out to exist in two competitors' docs. The retreat that followed positioned a leader like a challenger	The agent, then me
4	"Control of every monetization decision, judged on long-term revenue"; "monetization calls a PM can defend"	The first used Superwall's owned language and its reason to believe only worked for existing customers. The second was defensive, and came from one synthetic persona's worry rather than the PM's real pains	Three agents, then me
5	"Monetization that evolves alongside the product"; "make more money" and "long-term revenue"	The first is a pace comparison and a process, not an outcome. The second is the homepage's promise, not the persona's job	I did
✓ Kept	Monetization is a product decision. RevenueCat lets product managers make it like one	Survived all three agents as structure. Failed as copy, so it stays internal	Validated by agents

✦ The AI challenged me

I wanted to add "autopilot," the system picking and shipping winners automatically. The AI pushed back on three counts: rollout is human-triggered in the product, Autopilot is a competitor's product name, and it contradicts the control idea. Its reframe was better: the machine recommends, the PM decides.

✓ An agent caught it

All three PM personas rejected "monetization is a product decision" as customer-facing language: a truism, a turf claim, and wrong for enterprise. But each one, unprompted, described one of our three jobs in their own words. That validated the structure and killed the phrase as copy. Unowned is not the same as resonant.

An honest note. The final idea is close to where round one started. What changed is everything underneath it: it now comes from the PM's own job, it's structured by their real pains, and it's differentiated by proof rather than by a clever phrase.

From seven pain points to three jobs

The seven pains a PM feels map onto three jobs and a foundation. Each became a section of the page.

The PM's pain	Job	What delivers it	Page section
Paywall changes queue behind engineering	Change it without a release	Paywalls, remote configuration, one-action rollout	4, 5
Not knowing which test result to trust	Learn what users value	Experiments with predicted 12-month LTV	5
Blind spots between acquisition and revenue	Learn what users value	Campaign tracking in web funnels, Apple Search Ads keyword filters in Charts	5, 6
Monetization problems spotted too late	Learn what users value	Paywall charts, realized LTV per customer, Benchmarks	5
One offer for every user	Fit the offer to every user	Targeting, funnels that branch	6
Store commission pressure	Fit the offer to every user	Web billing	6
Store policy changes and refund abuse	Foundation	Refund Control, Apple and Google API changes handled	7

4 The system: five agents and a context file

I built the page with a small review system rather than a chain of prompts, so every claim, headline and visual got checked by something that wasn't the thing that wrote it.

Agent	Its job	What it caught
pm-critic	Reads everything as a solo PM, a growth PM and an enterprise PM	Validated the three-job structure; surfaced the enterprise governance question and a list of product gaps. On the finished page: a clear win over the current one for the solo and growth PM, partial for enterprise
competitor-watch	Checks every claim against what competitors own, claim or merely have	Killed a Superwall headline I planted as a calibration test, without being told it was one. Flagged that v0.1's core idea was Superwall's language

Agent	Its job	What it caught
web-copy	Drafts and tightens landing-page copy, always with a version cut by 30%. Used here for line-level cuts only	Every cut it proposed was applied before the build: 14 cuts, about 85 words (11%)
brand-guard	Compares the built page with RevenueCat's visual grammar	Five fixes in phase 3B, all applied: a leftover CSS rule that broke the funnel layout; section 3 tabs that were the largest blue fill on the page; section 7 icon tiles that read as the old feature grid; testimonials reworked into one centered block; and a container narrowed from 1,200px to about 990px, RevenueCat's width. Final verdict on the finished page: reads as a RevenueCat page, with no contrast or color-rule failures
claim-auditor	Traces every claim to a public source	56 fails in total, including a final sweep of 48 strings that failed 4. Among them: a product name that doesn't exist publicly, a testimonial with no speaker, "one click" where the product says "one action," Figma designs that come in through a separate plugin, a Benchmarks band drawn at 25th to 75th percentile where RevenueCat uses 20th to 80th, and "win-back," which RevenueCat uses for users who have already left

The context file. CLAUDE.md holds the audience, the non-negotiables, the design tokens, the contrast rules and the color roles. Every session and every agent reads it first.

The repo. Public, with the assignment brief, my notes, and the RevenueCat animations, product images and feature-page captures I studied kept out of it. Every decision is logged in `docs/iteration-log.md`.

Context states intent. Tooling enforces it. CLAUDE.md was created containing an em dash, breaking its own rule against them. So the rule became a pre-commit hook that blocks the commit. Then the hook was logged as built before it existed, a mistake the AI in chat made and caught itself on a later pass. Now it's real, and it blocks.

I challenged the AI

What the agents missed: a page for product managers that never once said "product manager" in its visible copy. Four agents had reviewed it. None was asked to check who the page speaks to. I caught it on a read-through. Agents check what they're told to check.

5 Architecture and copy

Architecture decisions

- **The hero carries the idea; the sections carry the proof.**
- **Half static, half dynamic.** The current page is entirely static imagery, and visual storytelling is the first thing being evaluated.

- **One fictional app across every visual**, Tidelark, a premium AI morning-routine coach, so the page tells one story instead of four unrelated ones.
- **Zigzag layout** for the job sections, like RevenueCat's own feature pages.
- **"Learn more" links to the matching RevenueCat feature pages**, so a PM can go deeper.

The hero: 19 lines considered

Candidate	Verdict
"Your paywall shouldn't wait for a release."	The strongest felt pain, but table stakes, and close to Superwall's promise. Became section 4's headline
"Change the paywall today. See its 12-month value."	Two ideas with no link between them, and "change" is an activity, not a capability
"Test pricing like you test features."	No outcome, no benefit, no difference
"Ship the paywall that pays, not the one that converts."	A competitor claims revenue over conversion, and "pays" reads as a guarantee
"Own your paywall without waiting on engineering."	Superwall's positioning, planted on purpose to test whether competitor-watch would catch it. It did
"Launch a paywall today. Judge it on the next 12 months."	"Today" isn't true for a first paywall, and the first half sits on Superwall's promise
"Stop shipping paywalls that win the week and lose the year."	The sharpest idea, but it opens by telling the reader they're wrong. Became section 5: "Win the year, not just the week."
"Build the subscription experience your users stay for."	The right idea. "Paywalls" narrowed it, "subscription experience" widened it
"Build a subscription experience worth renewing."	✓ Kept. A subscription is renewal by definition, so "worth renewing" is the user-value idea in two words. Top combined score across the three PM personas
"Build the reason they renew."	The cheekiest line. Lands best once the argument is made. Became the closing line, and the hero test variant

I challenged the AI

The first three hero lines the AI offered were weak, and I said so. They satisfied the rules, short, accurate and unowned, but didn't lead with what the reader gets. The wordplay that won, a subscription being renewal by definition, was mine.

The hero animation

The first version had a cursor clicking three cards around a phone. The clicks changed nothing. The rebuild follows the hero's own sub, in about eight seconds: **Design** (the featured plan changes on the phone), **Test** (weekly against monthly, with B pulling ahead), **Roll out** (a bar fills to 100%, then "No app release.").

I challenged the AI

A cursor clicking cards that change nothing is decoration, not a demo. I also pushed for a paywall that looks like a real, premium app rather than two plan rows. Every click in the new loop visibly changes what the user sees.

Section 3, three versions

Three arrow rows linking to the sections below, which was a table of contents pretending to be a section. Then tabs with a before-and-after drag slider. Then side-by-side panels, each built on its own idea. **Change** shows who owns each step: before, Engineering and the App Store own most of the track; with RevenueCat, every step says "You," and it reaches "Live" first. **Learn** shows paying customers as dots: week one favors A, the predicted year favors B. **Fit** contrasts one paywall for everyone, in the app only, with the right offer for each audience, in the app and on the web. The slider went because it only earns its place when what it reveals is strong enough.

One detail I like: I asked for B's dots to stay lit. The builder kept them fading too, just far less than A's, because that's what the model says. Honest proportions beat a cleaner picture.

The skimmer test

A PM who reads only the headlines should get the whole argument.

1	Build a subscription experience worth renewing.
2	Trusted by teams at [logos]
3	Change faster. Learn what renews. Fit every user.
4	Your paywall shouldn't wait for a release.
5	Win the year, not just the week. The test ends. The learning doesn't.
6	Different users. Different offers. App or web.
7	Subscribe once. Unlocked everywhere.
8	Two ways to start.
Closing:	Build the reason they renew.

6 Build and verification

Stack. Static HTML, CSS and vanilla JavaScript, no framework, deployed on Vercel. One landing page doesn't need a framework, and fewer moving parts meant fewer ways to break the night before the deadline.

Phase	What got built	Checkpoint
1	Scaffold with locked copy, design tokens, the first simulator	Worked on the first attempt
1.5	RevenueCat's visual grammar, proof points, simulator v2	Colors sampled from RevenueCat's own pages
2	Product-manager copy, the section 3 tabs, the first hero loop, zigzag layout	Interactive modules verified by keyboard
3A	One shared paywall, simulator v3 with a forecast chart, the hero redesigned	Every illustrative number refit to published ranges
3B	The Charts module, visuals for sections 3 to 7, a brand-guard review	No placeholders left
4	A tighter hero, side-by-side section 3, a compact simulator, charts instead of tables, web billing	Mobile simulator nearly halved
5	A red team against the brief and the current page; section 7 rebuilt as the foundation; a final review by all three agents	48 claims swept, 4 fixed
5.1	Final fixes from the agents' ranked list, including moving the Pixelcut quote	Design frozen

The simulator, four versions

Version	What it did	What was wrong	Who caught it
v1	A toggle swapped one metric and the winner flipped	The tension only existed across two moments in time. The rollout message appeared on the user's phone, where users would never see it	The AI's review
v2	Both metrics on every card; the PM's panel separated from the user's phone; a real paywall; a one-time cursor demo	It showed what won, not why. The numbers were too small to feel real, and the decision too obvious	I did
v3	A forecast chart: the observed month solid, the predicted months dashed. B overtakes at month 4. A "why B wins" panel. A live traffic split on the phone	LTV averaged across every customer in the test read like a monthly figure, and the module was more than 2,100px tall on mobile	I did
v4	LTV per paying customer (\$59.52 against \$130.46) with the math printed under the chart; the "why" moved onto the chart lines; a two-step rollout; the cursor demo runs the whole sequence	Final. Mobile height down from 2,167px to 1,210px	

✦ The AI challenged me

My idea for v3 was a crossover: variant A leads for three months, then B overtakes. With annual versus monthly plans that's impossible, because annual plans are paid upfront, so B would lead from day one. Someone who knows the subscription data would spot it immediately. We switched the test to weekly versus monthly, one of the most debated pricing questions in subscription apps, and the story became true.

✓ An agent caught it

Even then, illustrative numbers still have to be plausible. The audit tested the model against RevenueCat's own published conversion and retention data and refit every input to those ranges, keeping the story: A leads early, B overtakes at month 4 and ends about 47% higher.

✦ The AI challenged me

I wanted the winner to roll out automatically once the forecast appeared, so visitors wouldn't have to click. Automatic rollout is autopilot, which the product doesn't do and which we had rejected on purpose. Instead, the cursor demo performs the rollout itself, so every visitor sees it happen, and the PM still decides.

Two gaps I pushed to close in the final pass. The page showed charts as tables, so the Charts module now uses real line charts under RevenueCat's own chart names. And web billing, one of the most important new capabilities, was barely mentioned. It now has its own lines in section 6, in RevenueCat's own public phrasing, with no fee percentages.

Brand: rebuilt, not reused

I studied seven RevenueCat feature pages and two of their product animations, then rebuilt the visual grammar in code rather than reusing their files. The work had to be my own, and the reviewers would recognize their own assets. In production, I'd pull components from the design library and brief the agency for anything new.

- Colors sampled pixel by pixel from RevenueCat's pages: off-white #F9F9FB, a lavender and peach hero wash
- Color roles: blue for actions, green for winning and live states, red for emphasis
- RevenueCat's fonts are commercially licensed, so the page uses Manrope for display and RevenueCat's own declared fallback stack for body text

✓ An agent caught it

Three accessibility failures in the published brand palette: white text on brand red, brand green on white, and brand blue as link text on the off-white. Red moved off buttons, matching the live site, which already uses blue. Green became a fill instead of text. Links moved to a darker blue that passes everywhere.

Verified: 390px first, then 1440px, in a headless browser whenever the preview wasn't visible. Keyboard operation, screen reader announcements for state changes, focus handling, reduced motion, and every outbound link returning a valid page. Bugs caught and fixed include a style rule that showed both paywall states at once, compound words breaking at their hyphens on mobile, and a toast that covered the status line.

The red team: the new page against the current one

Before the last pass, I stepped back and tested the page against the brief and against the current /for-product page, side by side, the way a reviewer would.

	The current page	The new page
Promise	"You ship what matters, we handle monetization." Hand it off	"Build a subscription experience worth renewing." Own it
The PM's role	Bystander: RevenueCat saves engineering time	Operator: change it, learn from it, fit it
Structure	Seven feature tiles, then SDK setup steps	Three jobs drawn from the PM's pains; the headlines alone tell the story
Proof	One CTO quote about engineering time	Pixelcut, Floga, OpenAI, and RevenueCat's scale: 146K+ apps, \$17B+ processed
Visuals	Static screenshots	A hero loop, a simulator where the winner flips, live charts, a web-to-app journey
New capabilities	One A/B testing tile	AI Editor, predicted 12-month LTV, one-action rollout, Funnels, web billing, Benchmarks

What it found, and what changed:

- **Section 7 broke the story.** "Decide who can change a live paywall" was a governance feature that repeated a section 4 bullet, next to a visual about something else. It became the foundation: "Subscribe once. Unlocked everywhere."
- **Two truths from the current page had gone missing:** Customer Center, and Apple and Google API changes handled for you. For a page promising "worth renewing," losing the retention feature was a real gap. Both came back in section 7.
- **One pain had no home.** Blind spots between acquisition and revenue now map to campaign tracking in web funnels.
- **Density.** The web-to-app journey went from seven steps to four, at half the height, and tabs stopped scrolling sideways on mobile.

Then all three agents reviewed the finished page. pm-critic called it a clear win over the current page for the solo and growth PM, and a partial one for the enterprise PM, which led to a security and compliance link on the Enterprise card. brand-guard found no contrast or color-rule failures. claim-auditor swept 48 strings, failed 4, and all 4 were fixed.

✓ An agent caught it

Pixelcut's quote sat right beside the simulator, celebrating a 16% increase in subscribers, and its case study credits a weekly plan. So the proof point was praising exactly the kind of early win the simulator argues can mislead, and anyone at RevenueCat who knows that case study would have seen it. The quote moved to section 4, where it proves fast iteration instead of undercutting the forecast.

7 What I'd do next

The first three tests

Test	Hypothesis	Primary metric	Guardrail
Hero: "worth renewing" against "the reason they renew"	The clearer line wins with cold traffic	Sign-up rate from the page	Bounce rate
Simulator auto-demo on against off	Seeing the winner flip once, unprompted, carries more readers to pricing	Share reaching section 8	Time to first interaction
The link for existing customers	Many PM visitors already have RevenueCat installed and don't know Paywalls and Experiments are available	Clicks to the dashboard, then first paywall published	Sign-up rate

Questions I'd bring to Product

These came from pressure-testing the page with synthetic PM personas. They're hypotheses to validate with real customers, not findings.

- An approval step before a change goes live
- Reverting an offering or an experiment
- A holdout group after a winner rolls out
- A confidence range on the 12-month forecast, since credible intervals currently cover conversion metrics only
- Comparing past forecasts with what cohorts actually earned
- How long "enough data" takes for a low-traffic app
- Proactive alerts, since revenue anomaly emails are in beta

Small fixes I'd queue for the web team

The audit surfaced a few inconsistencies across RevenueCat's own pages. The kind of thing a PMM who owns the site fixes in week one:

- The AI Editor is labeled beta in the docs and generally available in the changelog

- The product demo linked from the Paywalls page predates the Figma import and the AI Editor, so it can't show either
- A Funnels page sentence reads "full control of price points on promotions," which looks like a typo
- The current /for-product page's demo dashboard link now lands on a login screen

With more time: a guided product demo

The simulator is a small version of what I'd build next: a guided demo a PM can play with before signing up. Pick your app category, build a paywall from a template, run a simulated test against benchmarks for similar apps, and roll out the winner. Five minutes, no account, no SDK. The current page's demo dashboard link now lands on a login screen; this would replace it with a guided version, aimed at PMs. It could take voice input, but voice isn't the point. The point is reaching the aha moment before the signup form.

One rule: every answer it gives comes from RevenueCat's own data and docs, never free-form AI. A demo that improvises will eventually promise something the product doesn't do.

How I'd measure it: demo completion, then signup rate from the demo, then first paywall published within a week.

A positioning idea for later

RevenueCat's scale lets it answer a question no challenger can: not "did B beat A," but "is my number any good?" Benchmarks shipped in April 2026. Bringing that context into every decision a PM makes is the strongest position I found that the product doesn't yet fully support.

A Appendix

Tools, and who did what

Tool	Role
Claude, in chat	Strategy partner: positioning, copy, red-teaming its own recommendations, reviewing agent output. Also extracted frames from RevenueCat's animations so the builder could study their motion
Claude Code	The builder: every file, the five agents, git, and verification in a headless browser
GitHub	Public repo, with the full decision history
Vercel	Hosting, redeployed on every push
Chrome DevTools and Font Finder	Typography and color sampling from RevenueCat's live site
Me	Every final decision

Assumptions

- 1 The page serves prospects and existing customers, and the mix is unknown. It leads for prospects and adds one link for customers.
- 2 The primary conversion is self-serve sign-up. Sales is secondary.
- 3 PMs own the product experience and strategy. Who owns the revenue number varies by company size.
- 4 Only public, shipped capabilities are claimed. Nothing from any roadmap.
- 5 All product data is illustrative and labeled. Tidelark is fictional.
- 6 RevenueCat's fonts are licensed, so the page substitutes.

What I couldn't verify

Real traffic, real customer interviews and internal data. The PM personas are synthetic, and their feedback is treated as hypotheses.

Sources

RevenueCat site and feature pages

revenuecat.com/
revenuecat.com/pricing/
revenuecat.com/for-product/
revenuecat.com/why-revenuecat
revenuecat.com/security-and-compliance
revenuecat.com/state-of-subscription-apps
revenuecat.com/feature/alerts-app-health
revenuecat.com/feature/charts
revenuecat.com/feature/experiments
revenuecat.com/feature/infrastructure
revenuecat.com/feature/paywalls
revenuecat.com/feature/refund-control
revenuecat.com/feature/web

Customer stories

revenuecat.com/customers/floga
revenuecat.com/customers/how-elevenlabs-hit-1m-arr-in-just-16-days
revenuecat.com/customers/mojo-case
revenuecat.com/customers/pixelcut
revenuecat.com/customers/revenuecat-openai

Changelog

revenuecat.com/changelog/release/audit-logs-2024-07-03
revenuecat.com/changelog/release/handling-refund-requests-consumption-requests-2024-11-13
revenuecat.com/changelog/release/rc-billing-apple-pay-google-pay-support-2024-11-29
revenuecat.com/changelog/release/charts-apple-search-ads-keyword-claim-type-2025-03-27

revenuecat.com/changelog/release/experiments-multivariate-testing-abcd-2025-04-29
revenuecat.com/changelog/release/track-paywall-performance-with-new-real-time-charts-2025-11-24
revenuecat.com/changelog/release/access-your-revenuecat-chart-data-via-api-2026-02-05
revenuecat.com/changelog/release/roll-out-an-experiment-winner-in-one-action-2026-02-06
revenuecat.com/changelog/release/see-predicted-12-month-ltv-winners-in-experiments-2026-02-12
revenuecat.com/changelog/release/see-who-edited-each-paywall-version-and-discard-unsaved-drafts-2026-02-16
revenuecat.com/changelog/release/compare-subscription-metrics-against-industry-benchmarks-2026-04-13
revenuecat.com/changelog/release/see-credible-intervals-in-experiment-results-2026-05-06
revenuecat.com/changelog/release/build-complete-web-to-app-funnels-with-revenuecat-2026-07-13
revenuecat.com/changelog/release/build-and-edit-paywalls-with-ai-2026-07-15
revenuecat.com/changelog/release/edit-paywall-rules-more-clearly-2026-07-30

Blog

revenuecat.com/blog/company/funnels-public-beta
revenuecat.com/blog/company/navigating-revenuecats-new-pricing-for-existing-users
revenuecat.com/blog/company/paywalls-on-the-web

Docs

revenuecat.com/docs/api-v2/charts-and-metrics
revenuecat.com/docs/customers/identifying-customers
revenuecat.com/docs/customers/refund-control
revenuecat.com/docs/dashboard-and-metrics/anomaly-detection-notifications

revenuecat.com/docs/dashboard-and-metrics/audit-logs
revenuecat.com/docs/dashboard-and-metrics/benchmarks
revenuecat.com/docs/dashboard-and-metrics/charts
revenuecat.com/docs/dashboard-and-metrics/charts/cohort-explorer
revenuecat.com/docs/dashboard-and-metrics/charts/paywall-conversion-chart
revenuecat.com/docs/dashboard-and-metrics/charts/paywall-ltv-chart
revenuecat.com/docs/dashboard-and-metrics/charts/prediction-explorer
revenuecat.com/docs/dashboard-and-metrics/reconciling-with-financial-reports
revenuecat.com/docs/getting-started/entitlements
revenuecat.com/docs/offerings/overview
revenuecat.com/docs/offerings/products-overview
revenuecat.com/docs/projects/collaborators
revenuecat.com/docs/projects/sso
revenuecat.com/docs/revenuecat-support/general-troubleshooting
revenuecat.com/docs/tools/experiments-v1
revenuecat.com/docs/tools/experiments-v1/configuring-experiments-v1

revenuecat.com/docs/tools/experiments-v1/experiments-overview-v1
revenuecat.com/docs/tools/experiments-v1/experiments-results-v1
revenuecat.com/docs/tools/funnels
revenuecat.com/docs/tools/funnels/configuring-payments
revenuecat.com/docs/tools/funnels/creating-funnels
revenuecat.com/docs/tools/funnels/deploying-funnels
revenuecat.com/docs/tools/paywalls
revenuecat.com/docs/tools/targeting
revenuecat.com/docs/web/overview
revenuecat.com/docs/web/paywalls
revenuecat.com/docs/web/redemption-links
revenuecat.com/docs/web/web-billing/localization
revenuecat.com/docs/web/web-billing/payment-methods
revenuecat.com/docs/welcome/set-up-revenuecat/account-management

Other

youtube.com/watch?v=mPzCTxIlMXE
apps.apple.com/us/app/stridewell/id6760530496